

Pipeline de données Big Data

Ingestion API → Data Lake (HDFS) → Spark/YARN → Data Warehouse
(PostgreSQL)

Orchestration Airflow • Docker • CI/CD GitHub Actions • AWS EC2

Rémi Descamps

Atelier : Déploiement de pipeline (Hadoop)

Dépôt du projet : github.com/rere1208/hadoop-pipeline

10 septembre 2026

1. Description du cas d'usage

Pipeline de données complet qui collecte, en continu, la météo et la qualité de l'air de plusieurs villes françaises via l'API publique **Open-Meteo** (données en temps réel, mises à jour en permanence — pas un jeu de données statique), stocke les données brutes dans un **Data Lake HDFS**, les nettoie et les transforme avec **Apache Spark** (soumis sur **YARN**), puis les charge dans un **Data Warehouse PostgreSQL** exposé via des vues SQL propres et analysables. L'ensemble est orchestré par **Apache Airflow** (planification quotidienne @daily, historique accumulé jour après jour), conteneurisé avec **Docker Compose**, testé et validé automatiquement via **GitHub Actions (CI/CD)**, et déployable sur **AWS EC2**.

Dépôt du projet (code source complet, README détaillé) :

<https://github.com/rere1208/hadoop-pipeline>

2. Architecture technique

Flux de données de bout en bout :

```
Open-Meteo API (meteo + qualite de l'air)
  | ingestion/extract_openmeteo.py
  v
JSON brut (local, horodate)
  | upload WebHDFS (etl/hdfs_utils.py)
  v
HDFS /data-lake/raw/weather/dt=YYYY-MM-DD/          <- Data Lake
  | spark-submit --master yarn --deploy-mode client
  | spark_jobs/transform_weather.py
  v
HDFS /data-lake/processed/weather/dt=YYYY-MM-DD/ (Parquet)
  | etl/load_to_postgres.py
  v
PostgreSQL (postgres-dwh)
  staging_weather -> vues clean_weather / daily_weather_summary  <- Data Warehouse
  ^
  +-- orchestre de bout en bout par Airflow (DAG @daily)
```

Composants Docker (réseau hadoop-net)

Service	Image / build	Rôle	Port hôte
namenode	bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-javab	HDFS NameNode	9870, 9000
datanode	bde2020/hadoop-datanode:...	HDFS DataNode	—
resourcemanager	bde2020/hadoop-resourcemanager:...	YARN ResourceManager	8088
nodemanager	bde2020/hadoop-nodemanager:...	YARN NodeManager	8042
historyserver	bde2020/hadoop-historyserver:...	YARN Timeline/History	8188
postgres-dwh	postgres:16	Data Warehouse	5433
postgres-airflow	postgres:16	Métadonnées Airflow	—
airflow-webserver	build local (airflow/Dockerfile)	UI Airflow	8080
airflow-scheduler	build local (airflow/Dockerfile)	Scheduler + driver Spark	—

L'image `airflow-spark (custom)` embarque Java 17, Spark 3.5.9 (build Hadoop 3) et les clients Python HDFS/PostgreSQL. Le job Spark est soumis en `--deploy-mode client` : le driver tourne dans ce conteneur, les exécuteurs sont lancés par YARN sur le nodemanager. La transformation n'utilisant aucune UDF Python, aucun interpréteur Python n'est requis côté exécuteur — ce qui évite d'installer Spark/Python dans les images Hadoop du cluster.

3. Prérequis

- Docker Desktop (avec WSL2 sous Windows) — Docker Engine ≥ 24 , Compose v2
- **16 Go de RAM minimum** allouée à Docker (stack HDFS+YARN+Spark+Airflow+Postgres gourmand)
- Ports libres sur l'hôte : 8080, 8088, 8042, 8188, 9870, 9000, 5433
- Un compte GitHub (dépôt) et, pour le déploiement, un compte AWS

Aucune installation locale de Java/Hadoop/Spark n'est nécessaire : tout tourne dans les conteneurs.

4. Installation et déploiement (local)

```
git clone https://github.com/rere1208/hadoop-pipeline.git
cd hadoop-pipeline
cp .env.example .env

docker compose up -d --build
```

Puis initialiser l'arborescence du Data Lake :

```
./scripts/init_hdfs_dirs.sh
```

Vérifier que tout tourne : `docker compose ps`. Pour un déploiement sur AWS EC2, voir `deploy/aws/DEPLOY.md` (scripts prêts, non exécutés pour ce rendu afin d'éviter les coûts AWS).

5. Guide d'exécution

Déclencher le pipeline

Via l'UI Airflow (<http://localhost:8080>) : activer puis déclencher le DAG `weather_pipeline_dag`. Ou en ligne de commande :

```
docker exec airflow-webserver airflow dags trigger weather_pipeline_dag
```

Vérifier chaque étape

- **Data Lake (HDFS)** : <http://localhost:9870> → Utilities > Browse the file system
- **Job Spark sur YARN** : <http://localhost:8088> → application avec l'état `FINISHED` / `SUCCEEDED`
- **Data Warehouse (PostgreSQL)** : `SELECT * FROM daily_weather_summary;`

6. CI/CD

Pipeline GitHub Actions (`.github/workflows/`) :

- **ci.yml** (automatique, à chaque push/PR sur main) : lint (ruff) → test (pytest) → build (image Docker)
- **deploy.yml** (déclenchement manuel uniquement) : déploiement SSH vers une instance EC2

Un pipeline GitLab CI équivalent (`.gitlab-ci.yml`) est également conservé dans le dépôt.

Pipeline exécuté et vérifié de bout en bout le 10 septembre 2026 : les 5 tâches du DAG Airflow se sont terminées avec succès, les 3 jobs Spark soumis sur YARN sont **FINISHED / SUCCEEDED**, et les données réelles de 5 villes françaises sont visibles, nettoyées et agrégées, dans le Data Warehouse.

[illegible]

NameNode HDFS — cluster actif, 1 DataNode live

data lake

Go

Search:

Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	Actions
☐	root	supergroup	0 B	Sep 10 10:41	0	0 B	processed	
☐	root	supergroup	0 B	Sep 10 10:40	0	0 B	raw	

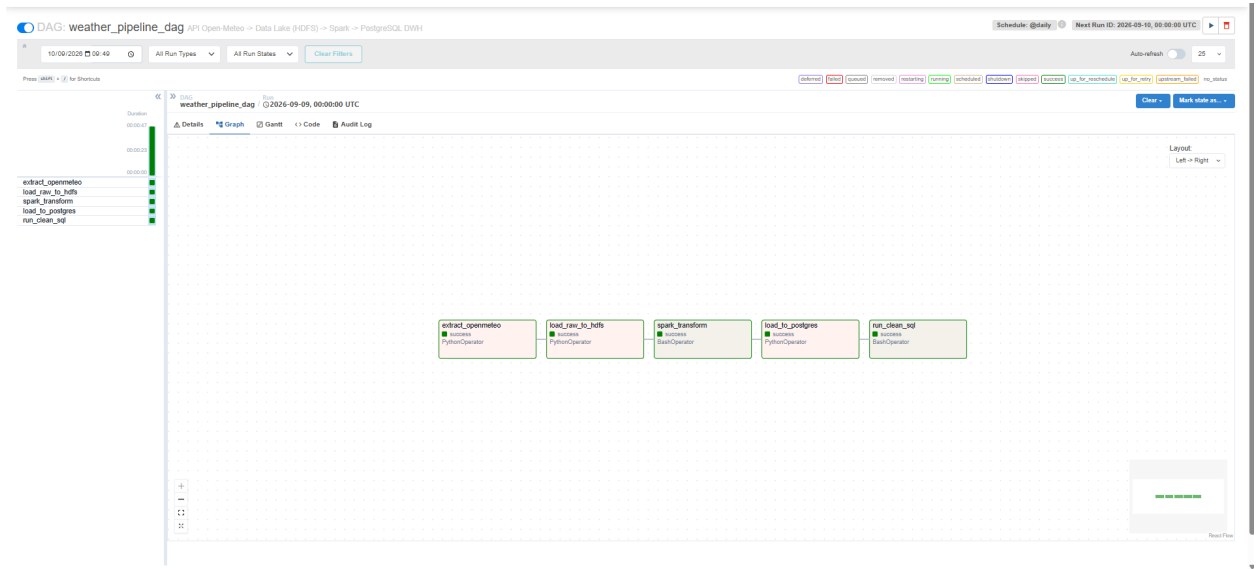
Showing 1 to 2 of 2 entries

Previous Next

Data Lake HDFS — dossiers raw/ et processed/

All Applications														
Application Name	Submission Time	Submission User	Submission Group	Submission Priority	Submission Queue	Submission Status	Submission Reason	Submission Location	Submission Size	Submission Memory	Submission CPU	Submission Network	Submission Storage	Submission Time
weather_pipeline_dag	2026-09-10 00:00:00	root	default	1	default	SUCCEEDED		YARN	100 MB	1 GB	100%	100%	100%	100%

YARN ResourceManager — 3 jobs Spark « weather-transform » FINISHED / SUCCEEDED



Graph du DAG weather_pipeline_dag — 5 tâches en success

city	reading_date	nb_readings	avg_temperature_c	min_temperature_c	max_temperature_c	avg_wind_speed_kmh	avg_european_aqi	dominant_air_qualit
ille	2026-09-10 00:00:00+00	1	14.7	14.7	14.7	8.6	30.0	fair
yon	2026-09-10 00:00:00+00	1	17.5	17.5	17.5	15.9	19.0	good
Marseille	2026-09-10 00:00:00+00	1	20.8	20.8	20.8	22.6	22.0	fair
Paris	2026-09-10 00:00:00+00	1	16.1	16.1	16.1	1.5	49.0	moderate
Toulouse	2026-09-10 00:00:00+00	1	16.2	16.2	16.2	6.3	21.0	fair

Résultat SQL — SELECT * FROM daily_weather_summary;

8. Arborescence du projet

```
hadoop-pipeline/  
  docker-compose.yml      services Hadoop, Spark client, Airflow, Postgres  
  hadoop.env              config du cluster HDFS/YARN (images bde2020)  
  airflow/  
    Dockerfile            image Airflow + Java + Spark + libs  
    conf/                 core-site.xml / hdfs-site.xml / yarn-site.xml  
    dags/weather_pipeline_dag.py  
  ingestion/extract_openmeteo.py  
  spark_jobs/transform_weather.py  
  etl/  hdfs_utils.py, load_to_postgres.py  
  sql/  01_create_schema.sql, 02_clean_views.sql  
  scripts/init_hdfs_dirs.sh  
  tests/  
  deploy/aws/  user-data.sh, DEPLOY.md  
  screenshots/  
  .github/workflows/  ci.yml, deploy.yml  
  .gitlab-ci.yml
```